

STL

vector 动态数组

```
vector<int> a; // 整数动态数组
a.push_back(0); // 向末尾插入一个数
a[0]; // 下标从0开始
n = a.size(); // 数组的长度，注意返回的数为无符号整型，不能直接做减法
a.clear(); // 将数组的长度归零
```

queue 队列

```
queue<int> q; // 整数队列
q.push(1); // 向队尾插入一个数
x = q.front(); // 取队首
q.pop(); // 删除队首
if (q.empty()) // 队列是否为空
```

priority_queue 优先队列（二叉堆）

```
priority_queue<int> q; // 整数优先队列
q.push(2); // 插入一个数
x = q.top(); // 取最大值
q.pop(); // 删除最大值
if (q.empty()) // 堆是否为空
```

stack 栈

```
stack<int> s; // 整数栈
s.push(2); // 插入一个数
s.top(); // 取栈顶
s.pop(); // 删除栈顶
n = s.size(); // 获取长度
s.empty() // 判断是否为空
```

set multiset 平衡树

```
// 相当于一个从小到大排序的序列，每次插入元素相当于会自动排序
set<int> s; // 元素不能重复的平衡树
multiset<int> s; // 元素能重复的平衡树
s.insert(3); // 插入一个元素，在set中，插入已有元素的操作会无效
s.erase(3); // 删除一个元素，在multiset中。会删除所有的该元素
```

```

set<int>::iterator it; // set的迭代器, 相当于访问set中元素的指针
multiset<int>::iterator it; // multiset的迭代器
it = s.begin(); // 获取第一个元素 (最小的元素)
it = s.end(); // 获取终止迭代器, 位于最后一个元素之后
it = s.lower_bound(3), it = s.upper_bound(2); // 见二分查找
it ++; // 让it指向下一个元素
it --; // 让it指向上一个元素
if(s.find(3) != s.end()) // 判断元素是否存在
s.erase(s.find(3)); // 用于multiset中删除一个指定元素
s.erase(it ++); // 用于删除it并且让it指向下一个元素
a = *it; // 取值
for(set<int>::iterator it = s.begin(); it != s.end(); it ++) // 遍历
for(auto &i : s) // 遍历所有数值

```

排序

```

sort(a + 1, a + n + 1); // 将a[1]到a[n]从小到大排序
sort(a + 1, a + n + 1, cmp) // 将a[1]到a[n]按照cmp指定的规则排序
// cmp(x, y)若返回1, 表示x排在y前面
// 注意不能 cmp(x, y) == cmp(y, x) == 1, 否则会运行错误

```

二分查找

```

// 使用二分查找前需要对数组排序
int *p = lower_bound(a + 1, a + n + 1, x); // 在a[1]到a[n]中找到大于等于x的第一个数的地址
int p = lower_bound(a + 1, a + n + 1, x) - a; // 获取下标
int *p = upper_bound(a + 1, a + n + 1, x); // 在a[1]到a[n]中找到大于x的第一个数的地址
int p = upper_bound(a + 1, a + n + 1, x) - a; // 获取下标

```

离散化

离散化指将n个任意大小的数映射为1到n的正整数, 并且保持相对大小不变。

```

int a[100001]; // 原数组
int c[100001], cnt = 0; // 用于离散化的数组, 和离散化后最大值的大小
for(int i = 1; i <= n; i ++) c[++ cnt] = a[i];
sort(c + 1, c + cnt + 1);
cnt = unique(c + 1, c + cnt + 1) - c - 1; // 去重, 可以跳过此步
for(int i = 1; i <= n; i ++) a[i] = lower_bound(c + 1, c + cnt + 1, a[i]) - c;
// c[a[i]] 即为 原始数组中下标为 i 位置的值

```

数学

gcd 最大公约数

```
int gcd(int x, int y) {
    while(x) x ^= y ^= x ^= y %= x;
    return y;
}
int gcd(int x, int y) {
    return x ? y : gcd(y, y % x);
}
```

power 快速幂

```
#define mod 998244353
long long power(long long x, int y) { // x 的 y 次方
    long long ans = 1;
    while(y) {
        if(y & 1) ans *= x, ans %= mod;
        x *= x, x %= mod;
        y >>= 1;
    }
    return ans;
}
```

预处理阶乘及逆元 求组合数

```
#define mod 998244353
long long fac[100001], inv[100001];
void init(int n) { // 预处理阶乘及逆元
    fac[0] = 1;
    for(int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % mod;
    inv[n] = power(fac[n], mod - 2);
    for(int i = n; i >= 1; i--) inv[i - 1] = inv[i] * i % mod;
}
long long C(int x, int y) { // 求组合数
    if(y > x) return 0;
    return (fac[x] * inv[y] % mod) * inv[x - y] % mod;
}
```

字符串

KMP 字符串匹配

```
int n;
char s[1000005];
int nxt[1000005] = {0};
```

```

void kmp() {
    int j = 0;
    for(int i = 2; i <= n; i++) {
        while(j && s[j + 1] != s[i]) j = nxt[j];
        if(s[j + 1] == s[i]) j++;
        nxt[i] = j;
    }
}

int m;
char a[1000005];
void work() {
    int j = 0;
    for(int i = 1; i <= m; i++) {
        while(j && s[j + 1] != a[i]) j = nxt[j];
        if(s[j + 1] == a[i]) j++;
        if(j == n) {
            // 此处以 i 为结尾找到了一次匹配
            // do something
            j = nxt[j];
        }
    }
}

```

读入

```

// 关闭同步, 加速 cin cout, 但是不能与 printf scanf 混用
ios::sync_with_stdio(0);

```

```

// 手动读数一个整数
int gi() {
    int x, c, op=1;
    while(c=getchar(), c<'0' || c>'9') if(c=='-') op=-op;
    x=c^48;
    while(c=getchar(), c>='0' && c<='9') x=(x<<3)+(x<<1)+(c^48);
    return x*op;
}

```

对拍

- `std.exe`: 程序1
- `bf.exe`: 程序2
- `data.exe`: 程序3
- `A.in`: 输入文件
- `A.out`: 程序1输出文件
- `A.ans`: 程序2输出文件

如果程序发生修改记得编译!

```
// judge.cpp 对拍程序
#include<cstdlib>
using namespace std;
int main() {
    while(1) {
        system("./data > A.in");
        system("./bf < A.in > A.ans");
        if(system("./std < A.in > A.out")) {
            puts("Runtime Error");
            break;
        }
        if(system("fc A.out A.ans")) {
            puts("Wrong Answer");
            break;
        }
        puts("Accepted");
    }
}
```

```
# judge.py 对拍程序python版
import os
while True :
    os.system("./data >A.in")
    os.system("./std <A.in >A.out")
    os.system("./bf <A.in >A.ans")
    if os.system("fc A.out A.ans") :
        print("WA")
        break
    print("AC")
```